

MOCCA lab2 作业报告

任务一 动作拼接处理

part 0

将 lab1 的代码复制后,发现缺了一个 channels 信息.按照 BVH 文件格式读入,对于 END 节点,channel 设置为 0,即可看到正常产生结果.

part 1

动作的插值.任务是将走和跑插值,形成一个类似于快走的动作.整体思路是先计算混合比例 α ,计算出比例后,按照比例对位置/速度进行 lerp,对角度/角速度进行 slerp.为了计算比例,需要先算出两个动作的速度,由于没有提供 fps 信息,将 fps 设定为 60.

接着计算 v_1 和 v_2 的实际速度值.利用目标速度 v ,计算出插值系数 α ,公式为 $(v - v_1) / (v_2 - v_1)$,并将其截断在 0 到 1 之间.然后根据 α 计算出生成动作的总帧数 n_3 .在新动作的逐帧生成中,将当前帧 i 按比例均匀映射到原始动作的帧索引 j 和 k ,对关节位置进行线性插值,对关节旋转调用 Slerp_rotation 进行球面线性插值,最终得到混合后的动作.

part 2

动作循环化任务是将一段首尾不完全一致的动作处理成可以无缝循环的动作.核心思路是利用阻尼弹簧模型,将首尾的姿态和速度差异平滑地分摊到整个动作周期中.

- 计算差值:计算首末两帧的 Root 节点位置差(忽略 Y 轴高度)以及速度差;同时计算首末两帧的旋转差和角速度差.
- 衰减分配:根据传入的 ratio 和 half_life 参数,利用 decay_spring_implicit_damping 函数,分别计算向前和向后的位置与旋转的偏移量.
- 叠加偏移:将生成的位置偏移直接加到原位置上,旋转偏移转换为四元数后与原旋转进行乘法结合,从而消除动作循环播放时的跳变现象.

part 3

动作平滑拼接.任务是将两段不同的动作拼接在一起,并在给定的过渡帧数内自然融合.

- 空间对齐:首先获取动作 1 在 mix_frame1 处的 Root 朝向(Y 轴旋转角)和 XZ 坐标.对动作 2 进行刚体变换(旋转和平移),使得动作 2 的第一帧能够接续动作 1 在混合开始时的位置和朝向.
- 过渡段混合:截取动作 1 从 mix_frame1 开始、长度为 mix_time 的片段,与对齐后的动作 2 的前 mix_time 帧进行融合.根据当前帧在过渡段中的时间进度 α ,对位置进行线性插值,对旋转进行 Slerp 插值.
- 截取与拼接:利用 sub_sequence 分别截取动作 1(混合前保留的部分)、生成好的融合过渡段、以及动作 2(混合后剩余的部分),最后按顺序使用 append 方法将这三部分拼接成一个完整的 BVHMotion 序列.

任务二 角色控制器设计 (Character Controller)

本部分的核心目标是基于已有的动作图 (Motion Graph) 和前置任务中实现的动作处理函数 (循环、拼接)，构建一个鲁棒的角色控制器，使其能够根据用户的期望轨迹 (输入的位置、旋转和速度) 实时、平滑地选择并播放合适的动画片段。

1. 整体架构与状态管理机制

控制器采用了一种“基于当前活动动作追踪” (Active Motion Tracking) 的策略。传统的简单控制器可能只在节点之间进行硬切换或简单的插值，而本设计中，维护了一个单一的 `active_motion` 变量。

- 在初始化时，控制器首先提取 Walk 节点 (Node 0)，并调用 `build_loop_motion` 将其处理为可以无缝循环的动作，作为角色的默认静息或直行状态。
- 为了提升运行效率并方便获取全局坐标，每次生成新的 `active_motion` 时，会通过 `batch_forward_kinematics` 预先计算好整个片段的所有关节全局位置和旋转 (`active_jt` 和 `active_jo`)，在后续的帧更新中直接查表提取。
- 控制器严格维护当前的全局 Root 状态 (`cur_root_pos` 和 `cur_root_rot`)，确保每一次新动作的接入都是基于当前物理世界的真实位置空间展开的，避免了由于动作片段自身的局部坐标导致角色“瞬移”。

2. 动作选择逻辑与代价函数 (Cost Function)

动作的选择依赖于对未来轨迹的预测与匹配。`_node_cost` 函数通过模拟未来的一小段时间，来评估候选动作与期望轨迹的契合度。

- **多时间点采样匹配：**不是仅仅对比下一帧，而是向后检查多个偏移量 (第 10、20、40 帧)。通过累加这些关键帧的误差，使得角色在选择动作时具有一定的“远见”。
- **权重分配策略：**
 - **旋转误差 ($W_{ROT} = 3.0$)：**赋予了最高权重。因为视觉上，角色的朝向错误比位置偏差更突兀。
 - **速度误差 ($W_{VEL} = 1.5$)：**次高权重，保证角色加减速的物理逻辑合理。
 - **位置误差 ($W_{POS} = 0.3$)：**权重最低，允许角色在位置上有微小的滑动以换取更平滑的姿态。
- **方向惩罚与步态奖励机制：**
 - **方向惩罚：**通过预计算的 `_yaw_delta` 获取每个 Turn 动作的固有偏转角。如果候选片段的转向与期望的偏航角 (Yaw Error) 方向相反 (即“南辕北辙”)，则施加强烈的惩罚 (`score += 4.0 * abs(nd)`)，避免角色做出不合逻辑的“反向旋转”。
 - **直行奖励：**为了避免角色在细微的轨迹变化中频繁触发极短的转向动画，对 Walk 节点施加了 `WALK_BONUS = 0.70` 的乘算衰减，使其在总代价值上具有天然优势，鼓励角色尽可能保持顺畅的直行。

3. 动作切换与平滑过渡 (Transitions)

控制器没有使用暴力的跨片段插值，而是深度利用了任务一中的 `concatenate_two_motions`。

- **物理融合过渡**：当决定切换到新节点时，利用 `_build_transition` 方法。该方法会将当前正在播放动作的剩余部分（Part 1）与新选中的目标动作（Part 2，已提前对齐到当前 Root 坐标）进行拼接。
- **混合窗口 (Mix Frames)**：设定了 `MIX_FRAMES = 12` 的混合窗口。通过将这 12 帧进行位置的线性插值和旋转的 Slerp 插值，生成了一段专属的过渡序列（Transition Sequence）。
- **退化保护 (Degeneration Fallback)**：如果当前动作的剩余帧数不足以支撑 12 帧的混合过渡（例如上一动作即将播放完毕），则触发 Fallback 机制，直接将新动作对齐到当前 Root 并使用传统的逐帧 Slerp 进行硬切过渡，确保程序不会崩溃或产生长度异常的 BVH 数据。

4. 状态防抖与稳定性优化 (Anti-jittering)

为了让角色的运动在视觉上看起来坚定且自然，在 `update_state` 中加入了两道“防抖”锁：

- **死区阈值 (Turn Threshold)**：设定 `TURN_THRESHOLD_RAD` 为 20 度。只有当期望朝向与当前朝向的偏差大于 20 度时，才允许退出默认的 Walk 状态去搜索 Turn 节点，过滤掉了输入轨迹中由于手部微操带来的高频抖动。
- **最小播放帧数 (Min Play Frames)**：设定 `MIN_PLAY_FRAMES = 20`。一旦角色开始执行某个转向动作，即便下一帧轨迹突然改变，也强制要求该动作至少播放 20 帧才能被高优先级的其他动作打断。这有效避免了角色在两个不同动作之间反复横跳（抽搐）的现象。

视频见压缩包